Université d'Avignon Centre de Recherche et d'Enseignement en Informatique (CERI)

OPTION : Intelligence Artificielle

NLP/Bert — Rapport de TP
Enseignant : Yannick Esteve

THEME

Prédiction de notes d'hôtels

Réalisé par :

- Amrouche Melissa

- Master 1 Intelligence Artificielle
- UE Apprentissage Profond Et Stochastique
- UCE Approches Neuronales

Année universitaire 2025/2026

CENTRE D'ENSEIGNEMENT ET DE RECHERCHE EN INFORMATIQUE
ceri.univ-avignon.fr

Table des matières

1 Présentation du projet **3**
1.1 Objectif . . . 3
1.2 Données . . . 3
1.3 Distribution des notes — Déséquilibre des classes . . . 3
1.4 Pipeline général du projet . . . 4
1.5 Métriques d'évaluation utilisées . . . 4
1.6 Plan de travail — Complexité croissante . . . 4

2 Étape 1 — Baseline TF-IDF + Modèles classiques **5**
2.1 Principe et motivation . . . 5
2.2 Prétraitement du texte . . . 5
2.3 Vectorisation TF-IDF . . . 6
2.4 Pipeline TF-IDF complet . . . 6
2.5 Modèles entraînés . . . 7
2.5.1 Logistic Regression (régression logistique) . . . 7
2.5.2 LinearSVC (machine à vecteurs de support linéaire) . . . 7
2.6 Résultats sur la validation . . . 7
2.6.1 Détail par classe — Logistic Regression (validation) . . . 7
2.7 Résultats sur le jeu de test . . . 8
2.8 Analyse des erreurs . . . 8
2.9 Observations et limites . . . 8

3 Étape 2 — Sentence Transformers **8**
3.1 Principe et motivation . . . 8
3.2 Prétraitement — Texte brut . . . 9
3.3 Pipeline Sentence Transformers complet . . . 9
3.4 Résultats sur la validation . . . 9
3.4.1 Détail par classe (validation) . . . 10
3.5 Résultats sur le jeu de test . . . 10
3.6 Analyse comparative TF-IDF vs Sentence Transformers . . . 10
3.7 Observations . . . 10

4 Étape 3 — Fine-tuning RoBERTa **11**
4.1 Principe — Qu'est-ce que le fine-tuning ? . . . 11
4.2 Architecture détaillée de RoBERTa pour la classification . . . 11
4.2.1 Le token [CLS] — Classification Token . . . 12
4.2.2 Multi-Head Self-Attention — Mécanisme d'attention . . . 12
4.3 Prétraitement — Tokenisation . . . 12
4.4 Dataset PyTorch et DataLoader . . . 12
4.5 Optimiseur, scheduler et hyperparamètres . . . 13
4.6 Boucle d'entraînement . . . 13
4.7 Courbes d'apprentissage . . . 14
4.8 Résultats finaux — Meilleur checkpoint (époque 2) . . . 15
4.8.1 Validation . . . 15
4.8.2 Test . . . 15
4.9 Analyse des erreurs . . . 15
4.10 Observations . . . 15

5 Comparaison générale des approches **16**

5.1 Tableau récapitulatif complet 16
5.2 Progression du F1 macro par classe 16
5.3 Analyse de la progression 16
5.3.1 Deux paliers de performance distincts 16
5.3.2 La représentation est le facteur dominant 16
5.3.3 Le déséquilibre des classes est le problème non résolu 17
5.4 Comparaison par type d'approche 17

Conclusion **17**

1 Présentation du projet

1.1 Objectif

L'objectif de ce projet est de prédire automatiquement la note attribuée par un client (de 1 à 5 étoiles) à partir du seul texte de son commentaire sur un hôtel, sans connaître la note à l'avance.

Il s'agit d'un problème de **classification de texte supervisée** en **5 classes** dans le domaine du **NLP** (Traitement Automatique du Langage). La classification est dite *supervisée* car le modèle apprend à partir d'exemples déjà étiquetés (texte + note connue), et *multiclasse* car il y a 5 catégories distinctes à prédire (et non 2 comme en classification binaire).

1.2 Données

Les données sont réparties en trois fichiers CSV, chacun ayant un rôle précis :

Fichier	Usage	Exemples
`train_hotel_reviews.csv`	Entraîner les modèles	18 491
`valid_hotel_reviews.csv`	Évaluer pendant le développement	1 000
`test_hotel_reviews.csv`	Évaluation finale unique	1 000

Table 1 – Répartition des données

Chaque ligne contient deux colonnes :

- `Review` : texte libre du commentaire (en anglais)
- `Rating` : note entière de 1 à 5 étoiles

La règle fondamentale respectée tout au long du projet : le jeu de **test** n'est consulté qu'une seule fois à la toute fin, après avoir figé le modèle définitif. Toutes les décisions de développement (choix des hyperparamètres, comparaison des modèles) sont prises sur le jeu de **validation**.

1.3 Distribution des notes — Déséquilibre des classes

Note	Nb exemples (train)	Proportion
1 ⋆	1 310	7.1 %
2 ⋆	1 630	8.8 %
3 ⋆	1 982	10.7 %
4 ⋆	5 465	29.6 %
5 ⋆	8 104	43.8 %
Total	**18 491**	**100 %**

Table 2 – Distribution des notes dans le jeu d'entraînement

Ce déséquilibre est le **défi principal** du projet : les notes 4 ⋆ et 5 ⋆ représentent à elles seules 73 % des données. Un modèle naïf qui prédirait systématiquement 5 ⋆ obtiendrait déjà 44 % d'accuracy (taux de bonnes prédictions) sans rien apprendre. Les classes 2 ⋆ et 3 ⋆ sont structurellement sous-représentées et donc difficiles à apprendre.

1.4 Pipeline général du projet

Voici la séquence complète depuis la donnée brute jusqu'à la prédiction finale :

```
Texte brut + note (CSV)
↓
Prétraitement du texte
(nettoyage ou texte brut selon l'approche)
↓
Représentation numérique du texte
(TF-IDF / Sentence Transformers / Tokenisation BERT)
↓
Modèle de classification
(LogReg / SVC / Réseau de neurones profond)
↓
Prédiction : note 1 à 5
↓
Évaluation : Accuracy + F1 macro
```

1.5 Métriques d'évaluation utilisées

Deux métriques complémentaires sont calculées sur chaque modèle :

Métrique	Définition
Accuracy (taux de bonnes prédictions)	Proportion de notes exactement correctes parmi toutes les prédictions. Accuracy $= \frac{\text{nb bonnes prédictions}}{\text{nb total}}$. Peut être trompeuse si les classes sont déséquilibrées.
F1 macro (moyenne harmonique précision/rappel, à poids égal par classe)	Pour chaque classe c, on calcule : $\text{F1}_c = \frac{2 \times P_c \times R_c}{P_c + R_c}$ où P_c (précision) = parmi les exemples prédits c, combien sont vraiment c ? et R_c (rappel) = parmi les vrais c, combien a-t-on correctement identifiés ? Le F1 macro est la moyenne de ces F1 sur les 5 classes — chaque classe compte autant, qu'elle ait 54 ou 484 exemples.

Table 3 – Métriques d'évaluation

Le **F1 macro** est la métrique de référence car elle pénalise les modèles qui ignorent les classes rares (2⋆, 3⋆), ce que l'accuracy seule ne détecterait pas.

1.6 Plan de travail — Complexité croissante

Trois approches sont comparées, classées par complexité croissante :

1. **TF-IDF + modèles classiques** — représentation par comptage de mots, classifieur linéaire
2. **Sentence Transformers** — embeddings pré-entraînés, classifieur linéaire
3. **Fine-tuning RoBERTa** — transformer profond entraîné sur nos données

Cette progression permet de quantifier le gain apporté par chaque niveau de complexité supplémentaire et de justifier l'usage d'architectures lourdes.

2 Étape 1 — Baseline TF-IDF + Modèles classiques

2.1 Principe et motivation

TF-IDF (*Term Frequency–Inverse Document Frequency* — Fréquence du terme pondérée par l'inverse de sa fréquence dans les documents) est la représentation de texte classique en NLP. Chaque commentaire est transformé en un vecteur numérique : chaque dimension correspond à un mot du vocabulaire, et sa valeur mesure l'importance de ce mot dans ce document.

La formule est :

$$\text{TF-IDF}(t, d) = \underbrace{\log(1 + f_{t,d})}_{\text{TF (fréquence du terme, avec lissage log)}} \times \underbrace{\log \frac{N}{|\{d : t \in d\}|}}_{\text{IDF (inverse de la fréquence documentaire)}} \tag{1}$$

où $f_{t,d}$ est le nombre d'occurrences du terme t dans le document d, et N le nombre total de documents. Un mot rare mais présent dans un avis (ex : "immaculate") obtient un score élevé. Un mot très fréquent partout (ex : "hotel") obtient un score faible.

Limites fondamentales de TF-IDF :

- **Pas de contexte** : "not good" et "very good" ont les mêmes dimensions actives, seuls les poids diffèrent — TF-IDF ne comprend pas la négation.
- **Pas de sémantique** : "great" et "excellent" sont deux dimensions entièrement distinctes, même s'ils signifient la même chose.
- **Représentation creuse** (sparse) : un vecteur de 20 000 dimensions dont la grande majorité est à zéro (un commentaire n'utilise que quelques centaines de mots différents sur 20 000 possibles).

2.2 Prétraitement du texte

Pour TF-IDF, un nettoyage **agressif** est appliqué avant la vectorisation. Ce nettoyage ne sera **pas** appliqué aux approches Transformers (qui gèrent le texte brut).

```
def clean_text(text):
    text = str(text)                        # Protection contre les
       valeurs NaN
    text = text.lower()                     # "Hotel" et "hotel" -> meme
        token
    text = re.sub(r'\d+', '', text)         # Supprimer les chiffres (
       peu informatifs)
    text = text.translate(str.maketrans( # Supprimer la ponctuation
        '', '', string.punctuation))
    text = re.sub(r'\s+', ' ', text).strip()  # Normaliser les espaces
    tokens = [w for w in text.split()
              if w not in STOPWORDS]        # Supprimer les stopwords
    return ' '.join(tokens)
```

Listing 1 – Fonction de nettoyage pour TF-IDF

Justification de chaque étape :

— **Minuscules** : “Hotel”, “hotel”, “HOTEL” → même token, évite la redondance.
— **Suppression des chiffres** : les numéros de chambre, années, prix varient et n’apportent pas d’information sur la satisfaction.
— **Suppression des stopwords** (mots très fréquents sans valeur informative : “the”, “a”, “is”, “in”) : ces mots apparaissent dans presque tous les documents, leur IDF est quasi nul — ils n’aident pas à discriminer les classes.
— **Limite connue** : la suppression de “not” peut inverser le sens (“not good” → “good”) — c’est une limite acceptée pour la baseline.

2.3 Vectorisation TF-IDF

Paramètre	Valeur et justification
`max_features=20000`	Garder les 20 000 termes les plus fréquents. Réduit la dimensionnalité, supprime les mots ultra-rares (fautes de frappe, noms propres) qui ne généralisent pas.
`ngram_range=(1, 2)`	Unigrammes (“good”, “clean”) ET bigrammes (“not good”, “very clean”). Les bigrammes capturent les négations et expressions figées que les unigrammes seuls ne peuvent représenter.
`sublinear_tf=True`	Applique $\log(1 + \text{tf})$ — atténue l’effet des mots très répétés. “clean” mentionné 10 fois ne vaut pas 10× “clean” mentionné 1 fois.
`min_df=3`	Ignorer les mots présents dans moins de 3 documents. Supprime les hapax (mots uniques, fautes) qui ne généralisent pas.

Table 4 – Paramètres du TfidfVectorizer

Règle fondamentale — pas de fuite de données (data leakage) : Le vectoriseur apprend le vocabulaire et les scores IDF **uniquement sur train**. Les jeux valid et test sont transformés avec ce vocabulaire figé. Si on calculait les IDF sur valid ou test, le modèle “verrait” des données futures et son évaluation serait artificiellement optimiste.

Résultat : matrices creuses de taille (18 491 × 20 000) pour train, (1 000 × 20 000) pour valid et test.

2.4 Pipeline TF-IDF complet

```
                    Texte brut
                        ↓
Nettoyage (lower + chiffres + ponctuation + stopwords)
                        ↓
        TfidfVectorizer.fit_transform(train)
         → Vecteur creux 20 000 dimensions
                        ↓
          LogisticRegression / LinearSVC
                        ↓
              Prédiction : note 1 à 5
```

2.5 Modèles entraînés

2.5.1 Logistic Regression (régression logistique)

La régression logistique est un modèle linéaire qui prédit la probabilité d'appartenance à chaque classe. Pour 5 classes, elle entraîne 5 classificateurs "one-vs-rest" (un par classe, chacun contre toutes les autres). Le paramètre `C=1.0` contrôle la régularisation L2 (pénalisation des poids trop grands pour éviter l'overfitting).

2.5.2 LinearSVC (machine à vecteurs de support linéaire)

LinearSVC cherche l'hyperplan qui sépare les classes avec la marge maximale. C'est un classifieur souvent plus rapide que la régression logistique sur du texte en haute dimension, mais il ne produit pas de probabilités — seulement des décisions.

2.6 Résultats sur la validation

Les resultats de : Accuracy (taux de bonnes prédictions) et (moyenne F1 par classe)

Modèle	Accuracy	F1 macro
Logistic Regression	**63.9 %**	**0.516**
LinearSVC	60.4 %	0.489

Table 5 – Résultats sur le jeu de validation — TF-IDF

La régression logistique surpasse LinearSVC malgré l'intuition contraire. Sur des données déséquilibrées, LinearSVC tend à favoriser les grandes marges qui profitent aux classes majoritaires, au détriment des classes rares.

2.6.1 Détail par classe — Logistic Regression (validation)

Classe	Précision	Rappel	F1	Support
1 ⋆	0.65	0.56	0.60	54
2 ⋆	0.39	0.35	0.37	82
3 ⋆	0.47	0.22	**0.29**	93
4 ⋆	0.52	0.52	0.52	287
5 ⋆	0.75	0.85	**0.79**	484
Macro avg	0.56	0.50	0.52	1000

Table 6 – Rapport de classification — Logistic Regression (validation)

Précision (parmi les avis prédits c, combien sont vraiment c ?) et **rappel** (parmi les vrais c, combien a-t-on correctement identifiés ?) sont les deux composantes du F1.

2.7 Résultats sur le jeu de test

Modèle	Accuracy	F1 macro
Logistic Regression	61.3 %	0.493

Table 7 – Résultats finaux sur le jeu de test — TF-IDF

2.8 Analyse des erreurs

Sur 1 000 exemples de validation, 361 sont mal classés (36.1 %). L'analyse des écarts montre que le modèle ne se trompe jamais grossièrement :

Écart entre note réelle et prédite	Nb erreurs
1 ⋆ (erreur légère)	304 (84 %)
2 ⋆ (erreur modérée)	44 (12 %)
≥3 ⋆ (erreur grave)	13 (4 %)

Table 8 – Répartition des erreurs par écart — Logistic Regression

2.9 Observations et limites

- La note 3 ⋆ (F1 = 0.29, rappel = 22 %) est la plus difficile : un avis mitigé contient des mots positifs ET négatifs, TF-IDF ne comprend pas leur équilibre.
- La note 5 ⋆ est la mieux prédite (F1 = 0.79) grâce à sa fréquence et à un vocabulaire très distinctif ("perfect", "excellent", "amazing").
- Le déséquilibre des classes est le principal facteur limitant : le modèle tire ses prédictions vers les classes fréquentes.
- **Ce score sert de baseline** (point de référence) : toute approche ultérieure doit dépasser 61.3 % d'accuracy et 0.493 de F1 macro pour être utile.

3 Étape 2 — Sentence Transformers

3.1 Principe et motivation

Sentence Transformers est une famille de modèles basés sur BERT (Bidirectional Encoder Representations from Transformers — représentations bidirectionnelles de transformers encodeurs) ré-entraînés pour produire un **vecteur dense unique** représentant le sens global d'une phrase entière.

Contrairement à TF-IDF qui compte les mots, Sentence Transformers **comprend le sens** grâce à un mécanisme d'attention bidirectionnelle : chaque mot est représenté en tenant compte de tous les autres mots de la phrase.

Modèle utilisé : `all-MiniLM-L6-v2`

- 6 couches Transformer (version allégée de BERT à 12 couches)
- Vecteurs de 384 dimensions (denses — toutes les valeurs sont significatives)
- Taille : ≈80 Mo — exécutable sur CPU sans GPU

— Pré-entraîné par apprentissage contrastif sur plus d'un milliard de paires de phrases : deux phrases de sens proche → vecteurs proches dans l'espace, deux phrases de sens opposé → vecteurs éloignés.

Critère	**TF-IDF**	**Sentence Transformers**
Représentation	Vecteur creux (20 000 dim)	Vecteur dense (384 dim)
Contexte	Aucun	Bidirectionnel
Sémantique	Non (« great » ≠ « excellent »)	Oui
« not good »	Similaire à « very good »	Vecteur opposé
GPU requis	Non	Non
Fine-tuning tâche	Non	Non

Table 9 – Comparaison TF-IDF vs Sentence Transformers

3.2 Prétraitement — Texte brut

Contrairement à TF-IDF, **aucun nettoyage agressif** n'est appliqué ici. Le modèle a été pré-entraîné sur du texte brut et a appris que :

— "not good" ≠ "good" (la négation change le sens — conserver "not")

— "Very clean !" ≠ "clean" (l'intensité et la ponctuation portent de l'information)

Supprimer les stopwords ou la ponctuation **dégraderait** les embeddings. Seule la gestion des valeurs nulles (NaN) et un `strip()` sont effectués.

3.3 Pipeline Sentence Transformers complet

```
Texte brut (strip uniquement)
↓
all-MiniLM-L6-v2 (modèle pré-entraîné, poids figés)
↓
Vecteur dense de 384 dimensions par avis
(sauvegardé en .npy pour éviter le recalcul)
↓
LogisticRegression (C=5.0)
↓
Prédiction : note 1 à 5
```

Pourquoi sauvegarder les embeddings en .npy ? Encoder 18 491 textes avec un transformer prend plusieurs minutes même sur CPU. Le format `.npy` (numpy binaire) permet de charger instantanément les vecteurs lors des exécutions suivantes du notebook.

Pourquoi C=5.0 au lieu de 1.0 ? Les embeddings (384 dimensions denses) sont beaucoup moins bruités que les vecteurs TF-IDF (20 000 dimensions creuses). La régularisation peut donc être réduite (C plus grand = moins de contrainte) car les features sont déjà bien structurées.

3.4 Résultats sur la validation

Modèle	**Accuracy**	**F1 macro**
Sentence Transformers + LogReg	63.1 %	0.510

Table 10 – Résultats sur le jeu de validation — Sentence Transformers

3.4.1 Détail par classe (validation)

Classe	Précision	Rappel	F1	Support
1⋆	0.70	0.57	0.63	54
2⋆	0.43	0.51	0.47	82
3⋆	0.27	0.11	**0.15**	93
4⋆	0.54	0.49	0.52	287
5⋆	0.73	0.84	**0.78**	484
Macro avg	0.53	0.51	0.51	1000

TABLE 11 – Rapport de classification — Sentence Transformers (validation)

3.5 Résultats sur le jeu de test

Classe	Précision	Rappel	F1	Support
1⋆	0.67	0.67	0.67	57
2⋆	0.51	0.53	0.52	81
3⋆	0.53	0.28	**0.37**	109
4⋆	0.51	0.50	0.51	287
5⋆	0.73	0.82	**0.77**	466
Macro avg	0.59	0.56	0.57	1000
Accuracy globale	**63.5 %** — F1 macro : **0.566**			

TABLE 12 – Rapport de classification — Sentence Transformers (test)

3.6 Analyse comparative TF-IDF vs Sentence Transformers

Classe	TF-IDF F1	SentTrans F1	Différence
1⋆	0.60	0.63	+0.03
2⋆	0.37	0.47	+0.10
3⋆	0.29	0.15	−0.14
4⋆	0.52	0.52	0.00
5⋆	0.79	0.78	−0.01

TABLE 13 – Comparaison F1 par classe : TF-IDF vs Sentence Transformers (validation)

3.7 Observations

- Sur la **validation**, Sentence Transformers est légèrement inférieur à TF-IDF (F1 macro : 0.510 vs 0.516). C'est un résultat paradoxal.
- Sur le **test**, Sentence Transformers surpasse nettement TF-IDF (F1 macro : **0.566** vs 0.493, soit +0.073), ce qui indique une **meilleure généralisation** — les embeddings sémantiques capturent des patterns plus robustes que les fréquences de mots.
- La note 2⋆ bénéficie le plus de l'approche sémantique (+0.10 de F1) : le modèle comprend le contexte négatif même quand les mots négatifs sont accompagnés de nuances.

— La note 3⋆ s'effondre (F1 = 0.15 sur valid, rappel = 11 %) : le modèle évite de prédire cette classe ambiguë et préfère systématiquement 2⋆ ou 4⋆. Cela s'explique par l'absence de fine-tuning : `all-MiniLM-L6-v2` est optimisé pour la similarité de phrases, pas pour la classification de sentiments graduée.
— **Limite fondamentale** : les deux approches (TF-IDF et Sentence Transformers) utilisent le modèle **sans l'adapter** à notre tâche. Le fine-tuning de la section suivante va remédier à cela.

4 Étape 3 — Fine-tuning RoBERTa

4.1 Principe — Qu'est-ce que le fine-tuning ?

Le fine-tuning (affinage) consiste à prendre un modèle pré-entraîné sur de très grandes quantités de données génériques, puis à le **ré-entraîner entièrement** sur nos données spécifiques. Tous les poids du modèle sont mis à jour — pas seulement une tête de classification ajoutée par-dessus.

Modèle utilisé : RoBERTa-base (Robustly Optimized BERT Pretraining Approach — approche d'entraînement BERT robustement optimisée)

— 125 millions de paramètres
— 12 couches Transformer (encodeur bidirectionnel)
— Vocabulaire BPE (Byte-Pair Encoding — encodage par paires d'octets) de 50 265 tokens
— Pré-entraîné sur 160 Go de texte (Wikipedia, livres, pages web)
— Amélioration de BERT : plus de données, meilleure stratégie de masquage, suppression des segments de phrases (plus simple et plus efficace)

4.2 Architecture détaillée de RoBERTa pour la classification

```
                    Texte brut
                        ↓
        Tokenizer BPE (Byte-Pair Encoding)
"unbelievable" -> ["un", "believ", "able"]
                        ↓
input_ids [CLS, t1, t2, ..., tN, SEP, PAD, PAD, ...] + attention_mask
              (max_length = 256 tokens)
                        ↓
           Embeddings (token + position)
                        ↓
          12 couches Transformer Encoder
Chaque couche : Multi-Head Self-Attention + Feed-Forward + LayerNorm
                        ↓
     Vecteur du token [CLS] (768 dimensions) CLS>CLS
= Classification token, token special en debut de sequence
        = resume le sens global de toute la phrase
                        ↓
      Dense(768 -> 768) + Dropout + Dense(768 -> 5)
= Tete de classification (initialisee aleatoirement, apprise par fine-tuning)
                        ↓
          Logits (5 scores bruts, un par classe)
                        ↓
Softmax -> Probabilites -> argmax -> Classe predite (0 a 4)
                        ↓
                +1 -> Note predite (1 a 5)
```

4.2.1 Le token [CLS] — Classification Token

Le token `[CLS]` (token de classification) est un token spécial inséré automatiquement **au début** de chaque séquence par le tokenizer. Après avoir traversé les 12 couches Transformer, ce token a été mis à jour par l'attention de tous les autres tokens de la séquence. Son vecteur final (768 dimensions) encode donc une représentation globale de toute la phrase, utilisée comme entrée de la tête de classification.

4.2.2 Multi-Head Self-Attention — Mécanisme d'attention

L'attention (Attention) est le mécanisme central des Transformers. Dans chaque couche, chaque token "regarde" tous les autres tokens de la séquence et décide combien de poids leur accorder pour construire sa propre représentation.

"Multi-head" (multi-têtes) signifie que ce mécanisme est appliqué en parallèle 12 fois avec des projections différentes, permettant de capturer différents types de relations entre les mots (syntaxe, sémantique, coréférence...).

4.3 Prétraitement — Tokenisation

Contrairement à TF-IDF, **aucun nettoyage manuel** n'est appliqué. Le tokenizer RoBERTa gère lui-même la normalisation.

```
tokenizer = RobertaTokenizer.from_pretrained('roberta-base')

def tokenize(texts):
    return tokenizer(
        texts,
        max_length=256,         # Tronquer a 256 tokens (limite memoire
            GPU)
        truncation=True,        # Couper les textes trop longs
        padding='max_length',   # Padder les textes courts a 256
        return_tensors='pt'     # Retourner des tenseurs PyTorch
    )
```

Listing 2 – Tokenisation pour RoBERTa

Pourquoi MAX_LEN=256 et pas 512 ? La mémoire GPU (VRAM) consommée par l'attention est quadratique avec la longueur de la séquence ($O(n^2)$) : doubler la longueur quadruple la mémoire. MAX_LEN=256 permet un batch de 32 exemples sur le GPU T4 (15.6 Go VRAM), contre un batch de 8 seulement avec MAX_LEN=512. La grande majorité des avis hôtels étant inférieurs à 256 tokens, la perte d'information par troncature est négligeable.

Décalage des labels : PyTorch `CrossEntropyLoss` (perte d'entropie croisée) exige des labels entiers de 0 à $N-1$. Les notes 1–5 sont donc converties en labels 0–4 (label = Rating − 1). À l'évaluation, on reconvertit : note = pred + 1.

4.4 Dataset PyTorch et DataLoader

Pour que PyTorch puisse itérer sur les données par mini-batches (petits lots), les données tokenisées sont encapsulées dans une classe `Dataset` :

```
class HotelDataset(Dataset):
    def __init__(self, encodings, labels):
        self.input_ids      = encodings['input_ids']
```

```
        self.attention_mask = encodings['attention_mask']
        self.labels = torch.tensor(labels, dtype=torch.long)

    def __getitem__(self, idx):
        return {
            'input_ids'      : self.input_ids[idx],
            'attention_mask' : self.attention_mask[idx],
            'labels'         : self.labels[idx]
        }
```

Listing 3 – Dataset PyTorch pour les avis d'hôtels

Le `DataLoader` (chargeur de données) utilise cette classe pour former des batches de taille 32, en mélangeant les exemples à chaque époque (`shuffle=True` sur train uniquement).

Pourquoi batch_size=32 ? Plus le batch est grand, plus les gradients sont stables (estimés sur plus d'exemples), et plus l'entraînement est rapide (meilleure utilisation parallèle du GPU). 32 est la valeur maximale qui tient en VRAM avec RoBERTa-base et MAX_LEN=256 sur T4.

4.5 Optimiseur, scheduler et hyperparamètres

Hyperparamètre	Valeur et justification
Optimiseur	AdamW (Adam avec Weight Decay découplé — régularisation L2). Standard pour le fine-tuning de transformers.
Learning rate	2×10^{-5}. Très petit pour préserver les représentations pré-entraînées. Un LR trop grand causerait le *catastrophic forgetting* (oubli catastrophique des connaissances acquises).
Weight decay	0.01. Pénalise les poids trop grands → réduit l'overfitting.
Époques	3. L'époque 2 donne le meilleur F1 valid ; l'époque 3 montre un léger overfitting (valid loss qui remonte).
Warmup	10 % des steps (≈173 steps sur 1 734 totaux). Le LR monte de 0 à 2×10^{-5} linéairement, puis redescend vers 0. Stabilise les premières étapes où la tête de classification est aléatoire.
Gradient clipping	Norme maximale = 1.0. Plafonne les gradients pour éviter les explosions de gradients, phénomène courant dans les transformers profonds.

Table 14 – Hyperparamètres RoBERTa-base

4.6 Boucle d'entraînement

À chaque époque, la boucle d'entraînement suit ce schéma sur chaque batch :

```
optimizer.zero_grad()
(reinitialiser les gradients accumules du batch precedent)
↓
Forward pass : model(input_ids, attention_mask, labels)
-> Calcul des logits (5 scores) + CrossEntropyLoss
(la loss mesure l'ecart entre la prediction et la vraie note)
↓
loss.backward()
(retropropagation : calcul du gradient de chaque parametre)
↓
clip_grad_norm_(model.parameters(), 1.0)
(ecreter les gradients trop grands)
↓
optimizer.step()
(mettre a jour les 125M parametres dans la direction opposee au gradient)
↓
scheduler.step()
(ajuster le learning rate selon le profil warmup + descente lineaire)
```

Après chaque époque, le modèle est évalué sur le jeu de validation. Le meilleur checkpoint (selon le F1 macro valid) est sauvegardé dans `best_roberta.pt`.

Pourquoi sauvegarder selon le F1 et non l'accuracy ? Sur des données déséquilibrées, l'accuracy peut augmenter même si les classes rares se dégradent (car les prédictions correctes sur 5⋆ compensent). Le F1 macro pénalise les mauvaises performances sur les classes minoritaires.

4.7 Courbes d'apprentissage

Époque	Train loss	Valid loss	Accuracy valid	F1 macro valid
1	0.9442	0.7918	65.8 %	0.6119
2	**0.6955**	**0.7038**	**69.4 %**	**0.6316** ← *sauvegardé*
3	0.6064	0.7205	69.9 %	0.6295

Table 15 – Courbes d'apprentissage — RoBERTa-base (3 époques)

Interprétation :

- **Époque 1** : le modèle apprend rapidement les patterns de base. La train loss (0.9442) est encore élevée — le modèle fait encore beaucoup d'erreurs.
- **Époque 2** : meilleur équilibre. La valid loss (0.7038) est à son minimum et le F1 macro valid (0.6316) est au plus haut → checkpoint sauvegardé.
- **Époque 3** : la train loss continue de baisser (0.6064) mais la valid loss remonte légèrement (0.7205) et le F1 valid baisse (0.6295). C'est le signe d'un début d'**overfitting** (sur-apprentissage) : le modèle mémorise les données d'entraînement au détriment de la généralisation.

4.8 Résultats finaux — Meilleur checkpoint (époque 2)

4.8.1 Validation

Classe	Précision	Rappel	F1	Support
1⋆	0.80	0.69	0.74	54
2⋆	0.52	0.62	0.56	82
3⋆	0.55	0.41	**0.47**	93
4⋆	0.59	0.54	0.57	287
5⋆	0.79	0.85	**0.82**	484
Macro avg	0.65	0.62	0.63	1000
Accuracy	**69.4 %** — F1 macro : **0.6316**			

Table 16 – Rapport de classification — RoBERTa-base (validation)

4.8.2 Test

Classe	Précision	Rappel	F1	Support
1⋆	0.69	0.75	0.72	57
2⋆	0.56	0.60	0.58	81
3⋆	0.61	0.35	**0.44**	109
4⋆	0.60	0.60	0.60	287
5⋆	0.81	0.86	**0.83**	466
Macro avg	0.65	0.63	0.64	1000
Accuracy	**70.4 %** — F1 macro : **0.636**			

Table 17 – Rapport de classification — RoBERTa-base (test) ← résultat final

4.9 Analyse des erreurs

Sur 1 000 exemples de validation, 306 sont mal classés (30.6 %). 94 % des erreurs ont un écart d'une seule étoile entre note réelle et note prédite — le modèle ne se trompe jamais grossièrement, uniquement sur les frontières floues (avis mitigés difficiles à distinguer entre deux notes adjacentes).

Confusions principales observées :

- Réel 3⋆ → prédit 4⋆ : erreur la plus fréquente. Un avis globalement satisfait avec quelques réserves peut sembler positif.
- Réel 4⋆ → prédit 5⋆ : frontière subjective entre "très bien" et "parfait".
- Réel 3⋆ : le rappel de 35 % signifie que 65 vrais avis mitigés sur 100 sont mal classés — c'est le point faible structurel du modèle.

4.10 Observations

- **Gain majeur** par rapport aux approches précédentes : +9.1 points d'accuracy et +0.143 de F1 macro par rapport à TF-IDF.

— **Le fine-tuning fait la différence** : Sentence Transformers (sans fine-tuning) plafonnait à 63.5 %. RoBERTa fine-tuné atteint 70.4 % — le même type de modèle mais adapté à la tâche spécifique.
— **La note 3⋆ reste le point faible** (F1 = 0.44, rappel = 35 %) : les avis ambigus sont intrinsèquement difficiles à classer, même pour un humain.
— L'overfitting apparaît dès l'époque 3 : 3 époques est le bon compromis pour ce dataset de taille modérée (18 491 exemples).

5 Comparaison générale des approches

5.1 Tableau récapitulatif complet

Approche	Acc. valid	F1 valid	Acc. test	F1 test
TF-IDF + LogReg	63.9 %	0.516	61.3 %	0.493
TF-IDF + LinearSVC	60.4 %	0.489	—	—
Sentence Transformers + LogReg	63.1 %	0.510	63.5 %	0.566
RoBERTa-base fine-tuné	**69.4 %**	**0.632**	**70.4 %**	**0.636**

Table 18 – Tableau comparatif de toutes les approches

5.2 Progression du F1 macro par classe

Classe	TF-IDF	SentTrans	RoBERTa	Progression
1⋆	0.62	0.67	0.72	+0.10
2⋆	0.39	0.52	0.58	+0.19
3⋆	0.18	0.37	0.44	+0.26
4⋆	0.49	0.51	0.60	+0.11
5⋆	0.78	0.77	0.83	+0.05

Table 19 – Évolution du F1 par classe (test) — de TF-IDF à RoBERTa

5.3 Analyse de la progression

5.3.1 Deux paliers de performance distincts

Les résultats révèlent deux paliers nets :

— **Palier 1 — Méthodes sans fine-tuning** (61 %–63 %) : TF-IDF et Sentence Transformers plafonnent dans cette zone malgré des approches fondamentalement différentes. La limite vient de l'absence d'adaptation à la tâche spécifique de classification de notes d'hôtels.
— **Palier 2 — Fine-tuning** (70 %) : RoBERTa fine-tuné franchit un seuil que les méthodes sans fine-tuning ne peuvent pas atteindre. L'entraînement sur nos données spécifiques permet au modèle de capturer des patterns propres aux avis hôteliers.

5.3.2 La représentation est le facteur dominant

Le gain le plus important provient non pas du changement de classifieur (LogReg vs SVC : +3 points), mais du changement de représentation :

— TF-IDF → Sentence Transformers : +2.2 points d'accuracy sur test
— Sentence Transformers → RoBERTa fine-tuné : +6.9 points d'accuracy sur test

5.3.3 Le déséquilibre des classes est le problème non résolu

La note 3⋆ reste le point faible de **toutes** les approches, progressant de F1=0.18 (TF-IDF) à F1=0.44 (RoBERTa). Bien que la progression soit significative (+0.26), le rappel de 35 % pour 3⋆ indique que 65 % des vrais avis mitigés sont encore mal classés. Les class weights (pondération des classes pendant la perte) n'ont pas été appliqués à RoBERTa dans cette version.

5.4 Comparaison par type d'approche

Critère	**TF-IDF**	**SentTrans**	**RoBERTa**
Représentation	Creuse, 20k dim	Dense, 384 dim	Dense, 768 dim
Contextuelle	Non	Oui	Oui
Fine-tuning tâche	Non	Non	Oui
GPU requis	Non	Non	Recommandé
Temps d'entraînement	Secondes	Minutes (CPU)	≈45 min (GPU T4)
Accuracy test	61.3 %	63.5 %	**70.4 %**
F1 macro test	0.493	0.566	**0.636**

Table 20 – Comparaison qualitative des approches

Conclusion

Ce projet compare trois approches de NLP pour la classification d'avis hôteliers en 5 classes de sentiment (1 à 5 étoiles), sur un dataset de 18 491 exemples d'entraînement fortement déséquilibré (44 % de notes 5⋆).

Résultats clés :

1. **RoBERTa-base fine-tuné** est le meilleur modèle avec 70.4 % d'accuracy et un F1 macro (moyenne harmonique précision/rappel à poids égal par classe) de 0.636 sur le jeu de test.
2. **La représentation est le facteur dominant** : le passage de TF-IDF à RoBERTa apporte +9.1 points d'accuracy, bien plus que tout changement de classifieur classique.
3. **Le fine-tuning est essentiel** : Sentence Transformers sans fine-tuning (63.5 %) plafonne nettement en dessous de RoBERTa fine-tuné (70.4 %), malgré une architecture similaire.
4. **La note 3⋆ reste le défi central** : avec un rappel de 35 %, les avis ambigus sont intrinsèquement difficiles à classer, même pour un modèle de 125 millions de paramètres.

Pistes d'amélioration identifiées :

— Fusionner les classes 3⋆ et 4⋆ pour passer à 4 classes distinctes. La note 3⋆ est structurellement ambiguë : ses avis partagent un vocabulaire proche de la note 4⋆, ce qui explique qu'elle soit la classe la plus mal prédite dans tous les modèles testés. Cette fusion réduirait mécaniquement les confusions et améliorerait les resultats.